

Efficient StreamingLLM with Attention Sinks

Group 6: Kshitij Ambilduke, Titouan Le Breton

1 Introduction to Streaming LLMs

The attention mechanism, now ubiquitous across domains, computes interactions between all pairs of tokens in a sequence. Processing T tokens therefore requires $\mathcal{O}(T^2)$ time and $\mathcal{O}(T)$ memory, a cost that becomes prohibitive in streaming settings where sequences grow unboundedly at inference time.

Dense attention and KV-caching. The standard formulation, which we refer to as *dense attention*, attends over the full history of T tokens. To avoid recomputing key and value projections at every step, a KV-cache stores them as they are produced, reducing the per-token cost to $\mathcal{O}(1)$ in time but retaining $\mathcal{O}(T)$ memory which grows without bound in a stream. A further complication is the context length mismatch problem. A model trained on sequences of length L encounters out-of-distribution positional encodings when evaluated on sequences of length $T \gg L$, causing perplexity to degrade beyond that point.

Sliding window attention without recomputation. A natural fix is to restrict the KV-cache to the most recent L tokens, discarding older entries as new tokens arrive. This bounds both memory and per-token latency at $\mathcal{O}(L)$, and keeps the attended context within the model’s training distribution. However, performance collapses once the sequence exceeds L tokens, due to two distinct failure modes: (a) **Context mismatch** Each cached key and value was computed in a context that included tokens now evicted from the window. These stale representations are therefore inconsistent with the current window, corrupting subsequent attention computations. (b) **Attention sink eviction.** Because softmax requires attention weights to sum to one, any attention mass the model does not productively allocate tends to concentrate on the earliest tokens in the sequence, which act as a passive “sink”. Evicting these tokens from the window removes this anchor, destabilising the attention distribution and sharply increasing perplexity.

Sliding window attention with recomputation. Both failure modes can be eliminated by recomputing attention from scratch at every decoding step, over the raw tokens in the current window rather than a stale cache. Positional encodings are assigned relative to position within the window, so the model always operates within its training distribution, and there are no cached representations to become inconsistent. This approach achieves strong perplexity on arbitrarily long sequences and serves as the quality gold standard in our experiments. Its cost, however, is $\mathcal{O}(TL^2)$ overall which is quadratic in L per token, making it impractical for large windows or long streams.

StreamingLLM. propose a sliding window variant that achieves the quality of recomputation without its computational overhead. The key observation is that only the attention sink tokens (a small fixed number of initial tokens) need to be permanently retained while the rest of the window can follow the standard sliding policy. RoPE positional embeddings are reassigned relative to position within this augmented window, eliminating the out-of-distribution encoding problem. Because the sink tokens are never evicted, both failure modes of the caching-based sliding window are resolved, while memory and per-token latency remain $\mathcal{O}(L)$.

The central claim of the authors is that *StreamingLLM matches the perplexity of the recomputation baseline while retaining the efficiency of the sliding window*. The remainder of this report evaluates that claim in a controlled experimental setting, examining both asymptotic behaviour and real-world system performance across model architectures.

2 Experimental Protocol

Our experiments compare all the four attention mechanisms discussed earlier across two recent lightweight language models: smolLM-360M and Qwen3-0.6B. These two models were chosen deliberately to span two important axes of variation. First, they differ in *architectural generation*: smolLM-360M is based on the Llama-2 architecture, which is representative of the class of models studied in the original StreamingLLM paper, while Qwen3-0.6B is a more recent architecture. This allows us to assess whether the paper’s claims generalise beyond the models on which they were originally validated. Second, they differ substantially in *context length and RoPE configuration*: smolLM-360M has a context length of 2048 tokens and a base RoPE period of 10k, whereas Qwen3-0.6B supports a context length of 32 768 tokens with a base RoPE period of 1000k. This contrast lets us probe how positional encoding range affects the behaviour of each attention mechanism under long-context inference.

Three metrics are used to evaluate the attention mechanisms: perplexity/NLL (Negative Log Likelihood), peak memory usage, and average token generation latency. The PG19 dataset, a collection of books, was used as the evaluation corpus, consistent with the original paper. Four initial tokens are retained as attention sinks in all relevant experiments.

The experiments were conducted across several window sizes, set to half the model context length, the full context length, and beyond the context length. This translates to window sizes of 1024, 2048, and 4096 for smolLM-360M with 20k total tokens and 16 384, 32 768, and 48 000 for Qwen3-0.6B, with 100k total tokens. Note that for Qwen3-0.6B, some dense attention and sliding window with recomputation runs could not be completed due to excessive computational cost; further details are given in Section 3.

All experiments were run on an RTX 5060 Ti with 16 GB of VRAM under Linux. A subset of the smolLM-360M experiments was additionally run on an RTX 4060 with 8 GB of VRAM under Windows 11, in order to assess the influence of hardware and operating system on latency and token generation cost. Further details are given in Section 3.

3 Results & Analysis

In this section, we present our results that pertain to the main claim of the article: that StreamingLLM manages to achieve the “gold standard” performance of sliding window with re-compute, all while keeping the better memory and time complexity of sliding window without recompute.

3.1 Memory and Latency

In our experiments, as expected, we did notice the linear increase of the memory cost of dense attention as shown in Figure 1. For Qwen3-0.6B, we only ran dense attention up to 40k tokens due to the ever increasing memory cost, and sliding recompute up to 16k tokens due to cost complexity (the elevated memory cost for sliding recompute is discussed in Section 3.3). The linear increase in memory cost makes dense attention not feasible for very long context lengths.

The results for the latency of sliding window with recompute are also as expected. On smolLM-360M, as the window size increases, the latency for token generation with sliding recompute also increases. For Qwen3-0.6B, the cost becomes so prohibitive that attention was only run with both a total window size and a total number of tokens of 16 384. The latency results can be seen in Figure 2.

For sliding window without recompute and attention sinks, the latency and memory values behave as expected. When the context window L is large, both have reasonable latencies compared to sliding recompute, and their memory footprint increases linearly with the sliding window

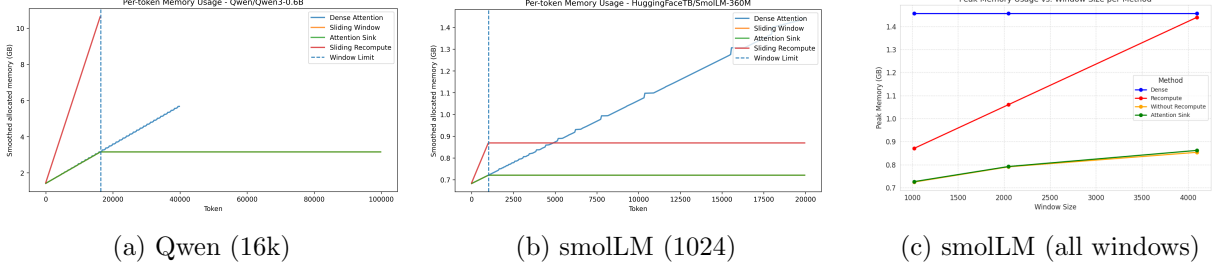


Figure 1: Memory cost of the attention mechanisms. Window size given in parenthesis.

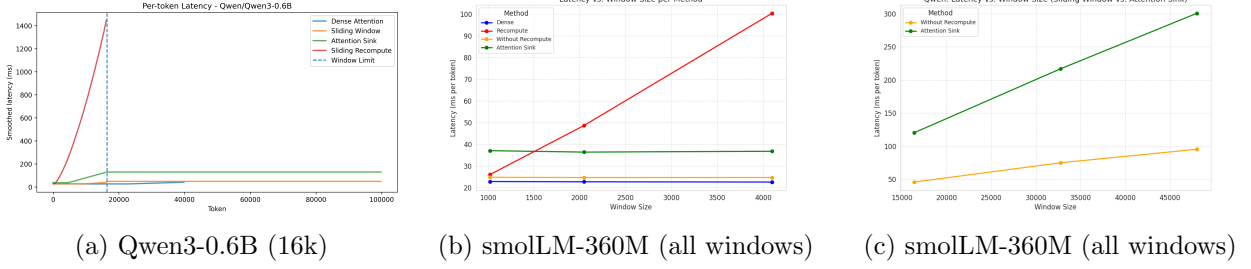


Figure 2: Latency of the attention mechanisms. Window size given in parenthesis.

size rather than with the overall number of tokens in the sequence, contrary to dense attention. Together, these results establish that sliding window without recompute and attention sinks are the only two mechanisms that remain viable for very long sequences; we therefore focus on their perplexity in the next section.

3.2 Perplexity

We show our results in Figures 3, 4, and ?. Across our experiments, the performance claims on the perplexity score of the different methods made in the original paper are largely validated by our experiments. The NLL score of window attention without re-computation does spike as soon as we go beyond the window size, and sliding window with re-computation does present a gold standard with the lowest perplexity. Attention sink does indeed achieve the same low perplexity as sliding window with re-computation.

Nevertheless, when it comes to the perplexity of dense attention and sliding window attention, and about when perplexity spikes relative to the model’s context window, we did notice some different behaviour between smolLM-360M and Qwen3-0.6B. These are further discussed in Section 3.3.

3.3 Further Results

Overall, our experiments confirm the main claims of the article, especially about asymptotic time and memory complexity, as well as the optimal perplexity of attention sinks. However, for specific claims, we didn’t always get the results we expected: these are discussed here.

The memory cost of sliding recompute was surprisingly high. It should have a similar peak memory usage than attention sink and sliding window without re-computation. In practice, memory cost were around 2-times higher on smolLM and 3 times higher on Qwen, as seen in Figure 1. In practice, such an increase in memory cost is very consequential, especially since VRAM is a hard limit: if the threshold is surpassed, the model can no longer run. This increase in memory cost is

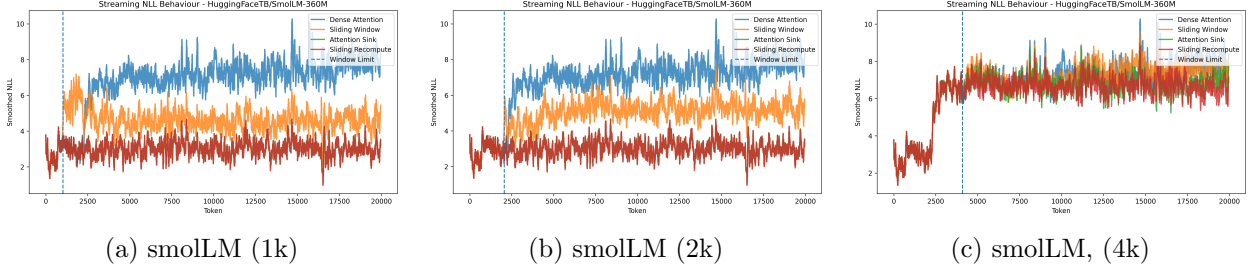


Figure 3: Perplexity for smolLM-360M across different window sizes (given in parenthesis).

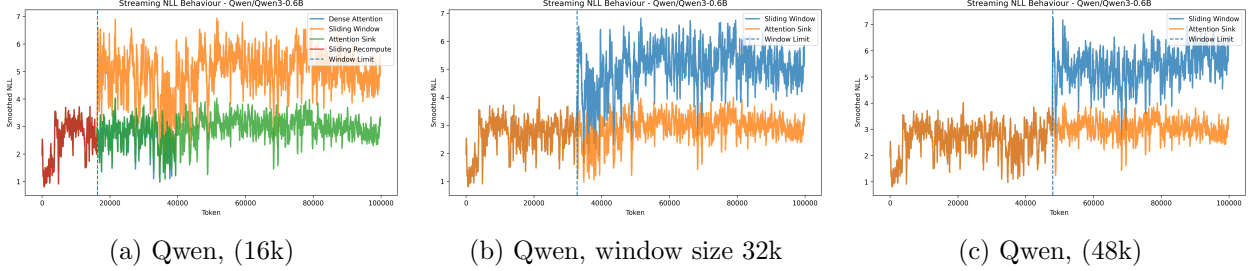


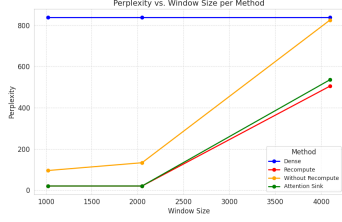
Figure 4: Perplexity evolution for Qwen3-0.6B across different window sizes (given in parenthesis).

due to the need to store intermediary states for later re-computation. For smolLM (where results are comparable because we ran all the experiments without computational limitations), sliding window with re-computation even reached a similar peak attention to dense attention.

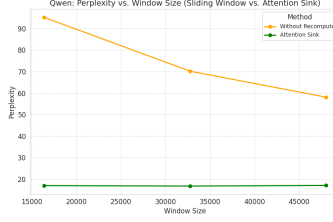
For latency value, there were major variations compared to what could be expected from the big-O costs. If we look at how average token latency changed with the context window on the smolLM experiments on rtx5060 (Figure 2), we see, as expected, that the sliding window with recompute (time cost of $\mathcal{O}(TL^2)$) increases much faster than that of sliding without recompute and attention sink (time cost of $\mathcal{O}(TL)$), where as the latency of dense attention doesn't change at all. However, it is uprisng to see that even for a window size of 1024, how low the latency of sliding window re-computation is, as well as how fast dense attention is. This is due to the code implementation: sliding window with re-computation as well as dense attention both rely on the default attention mechanism implementation of hugging-face, which has been optimized with custom fused kernels, where as sliding window without re-computation and attention sinks rely on custom python loops.

Another surprise on latency values is how attention sinks is consistently slower than sliding window, despite their time complexity being equal in theory (see Figure 2). This is due to the RoPE positional re-computations inside attention sink, which slow down the GPU computations significantly. This highlights the importance of implementing and relying on code that is GPU-friendly, and how asymptotic costs do not translate 1 for 1 into the true observed performance.

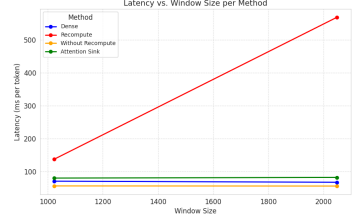
Surprised by how fast sliding recompute ran with a 1024 and 2048 cache size on smolLM with the rtx5060ti, the experiments were reproduced with an rtx 4060 with a windows OS. The results for perplexity and memory cost were the same, but the latency of each method change drastically. The results can be seen in Figure ?? (the experiments for rtx 5060 are in Figure 2). This experiments highlight once again how system-dependent performance is in practice. First, we observe that sliding window with re-compute and dense attention are now much slower than the other attention mechanisms now that it lacks the fused kernels optimized for linux. However, overall, all attention mechanisms are much slower. Part of this is due to the graphics card and the switch to a windows OS, but an overall slower computer system (with slower RAM for example) is likely also there to



(a) smolLM, all window sizes



(b) Qwen, all window sizes



(c) smolLM latency (RTX 4060)

Figure 5: Comparison of perplexity evolution across all tested window sizes for both models, and average token latency changes with the context window on smolLM.

blame. Once again, these experiments show that asymptotic performance, in practice, are only part of the performance consideration, and having actual code optimized for a computer system can also be decisive.

Interesting results were also observed relative to the behaviour of perplexity and context windows (see Figures 3, 4). The original paper claimed that dense attention performance would drastically fall once the context window of the model was exceeded. This was observed on our smolLM-360M experiments, where dense attention but also sliding window attention (when the window size was 4k, superior to the 2k context window) perplexity spiked once the threshold was crossed. However, for Qwen3-0.6B, neither of these spikes in performance were observed, indicating that advances in modern LLM training have managed to fix this defect of older models.

4 Conclusion

Through the experiments carried out, the main claim of the paper, that attention sink achieved state of the art performance on complexity while being less asymptotically costly than dense attention and sliding window with recomputes, has been validated. However, some important caveats have been found. The main distinction was around dense attention which the original article claimed that perplexity and latency were major issues. On our experiments with dense attention, latency was never an issue, and although perplexity was an issue with smolLM-360M, it was not one with Qwen3-0.6B. The main issue we encountered was the out-of-Memory errors due to a linear increase in memory (mentioned in the original article).

Our experiments also put forward the importance of system considerations, something the original article didn't mention substantially. Various results (memory usage of sliding recompute, latency of dense attention, latency of attention sinks due to RoPE, latency on rtx4060 vs rtx 5060ti) show the very significant impact having an optimized software can have. Even for relatively long context windows, a theoretically sub-optimal method can still be more efficient if the software has been optimized for it.